Auth assignments: sessions and Google sign-in

Oct 29, 2026 · @Ishan

Two small builds, about 20–25 minutes each with an AI assistant: a session login on MongoDB Atlas deployed live, and a Google sign-in using Passport.js.

Ground rules

- **Time:** about 45 minutes for both. Aim for 25 on Assignment 1 and 20 on Assignment 2.
- **AI is allowed.** Use it to write code, but you must be able to explain every security rule in your own words. You'll be asked about them live.
- **Keep it small.** Plain HTML forms rendered by Express are fine. No React, no styling marks. The security rules are what's graded.
- **Answer the "Why?" questions** in a `SECURITY.md` file in your repo, one or two sentences each. Write them yourself, not with AI.
- **Never commit a secret.** If one reaches GitHub, even for a minute, treat it as leaked: rotate it.

Assignment 1 · Session login on MongoDB Atlas (25 min)

Build a tiny site where users sign up, log in and see a private page. The session lives in your database, and the site is live on the internet.

Build

1. Pages: `/signup`, `/login`, `/dashboard` (only when logged in) and a **Log out** button (`POST /logout`).
2. The dashboard shows the user's name and email, plus their session's created and expiry times.
3. Store data in a free **MongoDB Atlas** cluster, using two collections: `users` and `sessions`.
4. Write the session logic yourself in Express, without `express-session`. It's about 40 lines and it's the point of the exercise.
5. Deploy to **Vercel** (export your Express app with `export default app`) or **AWS** (App Runner, Elastic Beanstalk or EC2). Your choice.

Atlas setup

1. Create a free M0 cluster.
2. **Database Access:** create a user with `readWrite` on *your app's database only*, with a generated password.
3. **Network Access:** allow `0.0.0.0/0`. Vercel's IP addresses change, so you can't list them.
4. Copy the `mongodb+srv://` connection string into `MONGODB_URI`: in `.env` locally, and in your Vercel or AWS environment settings.

Security rules

#	Rule	Why? (answer in SECURITY.md)
1	Hash passwords with `bcrypt`, cost 10 or more.	What does bcrypt's salt protect against?
2	Make the session id with `crypto.randomBytes(32)`.	Why not use the user's `_id` or `Date.now()` as the id?
3	Create a new session id on every login.	What is session fixation?
4	Session cookie is `HttpOnly`, `Secure` and `SameSite=Lax`.	What does each of the three flags block?
5	Sessions expire after 30 minutes: an `expiresAt` field plus a TTL index.	What happens to a stolen cookie after expiry?
6	Logout deletes the session document.	Why isn't clearing the cookie enough?
7	Same error for a wrong email and a wrong password.	What would an attacker learn otherwise?
8	`MONGODB_URI` lives only in env vars, and `.env` is in `.gitignore`.	What could someone do with your connection string?
9	The Atlas user can only `readWrite` your one database.	Why not connect as an admin user?

Two more questions

- Your cluster accepts connections from `0.0.0.0/0`. What still keeps strangers out?
- Why would storing sessions in server memory, instead of Atlas, break on Vercel?

Prove it works: log in on your live site. Then delete your session document in Atlas and refresh. You should be logged out.

Assignment 2 · Google sign-in with Passport.js (20 min)

Build a login page with one button, **Continue with Google**. Your app never sees a password: Google checks it, and then your app starts its own session.

Build

1. `/` shows the **Continue with Google** button, which links to `/auth/google`.
2. `/auth/google/callback` sends the user to `/profile`, which shows their name, email and photo from Google.
3. A **Log out** button (`POST /logout`).
4. Use `passport`, `passport-google-oauth20` and `express-session`. Running on `http://localhost:3000` is enough; deploying is a bonus.

Google Cloud Console setup

1. Create a project at console.cloud.google.com.
2. Open **Google Auth Platform**. Under Branding, fill in an app name and your email. Under Audience, choose **External**, keep it in **Testing**, and add your own Gmail as a test user.
3. Under **Clients**, create an OAuth client of type **Web application**:
   - Authorized JavaScript origin: `http://localhost:3000`
   - Authorized redirect URI: `http://localhost:3000/auth/google/callback`
4. Put the client ID and secret in `.env` as `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`, along with a long random `SESSION_SECRET`.

Security rules

#	Rule	Why? (answer in SECURITY.md)
1	The client secret and `SESSION_SECRET` live only in `.env`, never in browser code.	Why is the client ID fine to show but the secret isn't?
2	Request only the `profile` and `email` scopes.	Why ask Google for as little as possible?
3	Register the exact callback URL, with no wildcards.	What could an attacker do if Google redirected anywhere?
4	Turn on the `state` check (`state: true` in the strategy options).	Which attack does `state` stop?
5	Identify users by Google's `profile.id`, not their email.	Why is an email a weak unique key?
6	Session cookie is `HttpOnly` and `SameSite=Lax`, not `Strict`.	Why would `Strict` break the redirect back from Google?

#	Rule	Why? (answer in SECURITY.md)
7	Logout is a `POST` that calls `req.logout()` and destroys the session.	Why shouldn't logout be a plain GET link?
8	Use Passport 0.6 or later.	What does it do to the session id at login, and why? (Hint: rule 3 of Assignment 1.)

Two more questions

- In this app, who checks the user's password, and what does your server receive instead?
- After the Google redirect, how does your app remember the user on the next request?

Prove it works: sign in and see your profile. Then open `/profile` in an incognito window. You should be sent back to `/`.

Find the SHA

SHA-256 turns any input into a fixed 64-character fingerprint, and it can't be reversed. It's fast, which is good in some places and dangerous in others. Answer these in `SECURITY.md`.

1. **Session ids at rest.** Assignment 1 stores the raw session id in Atlas. If your database leaked, what could an attacker do with it? How would storing `sha256(sessionId)` instead fix that, and what changes in your login and lookup code? *Bonus: implement it.*
2. **Fast vs slow.** SHA-256 is fine for a session id but wrong for passwords, where we use bcrypt. Why? Think about how random each input is and how many guesses per second an attacker can make.
3. **Google's ID token.** Its header says `alg: RS256`. Where is SHA-256 in that, and what does the signature prove to your server?
4. **Your HTTPS padlock.** Open your deployed site, click the padlock and view the certificate. Find SHA-256 in it. What is it used for?
5. **Spot the mistake.** A teammate suggests `sha256(password)` to "encrypt" passwords before saving them. Name two things wrong with that sentence.

Submit

- [ ] One GitHub repo with two folders: `session-atlas/` and `google-oauth/`
- [ ] Live URL for Assignment 1
- [ ] `.env.example` in each folder, with placeholder values only

- [ ] `SECURITY.md` answering every "Why?", the two extra questions per assignment, and Find the SHA
- [ ] A screenshot of your Atlas `sessions` collection with one live session
- [ ] Be ready to demo live: log in, delete the session in Atlas, refresh, and get logged out